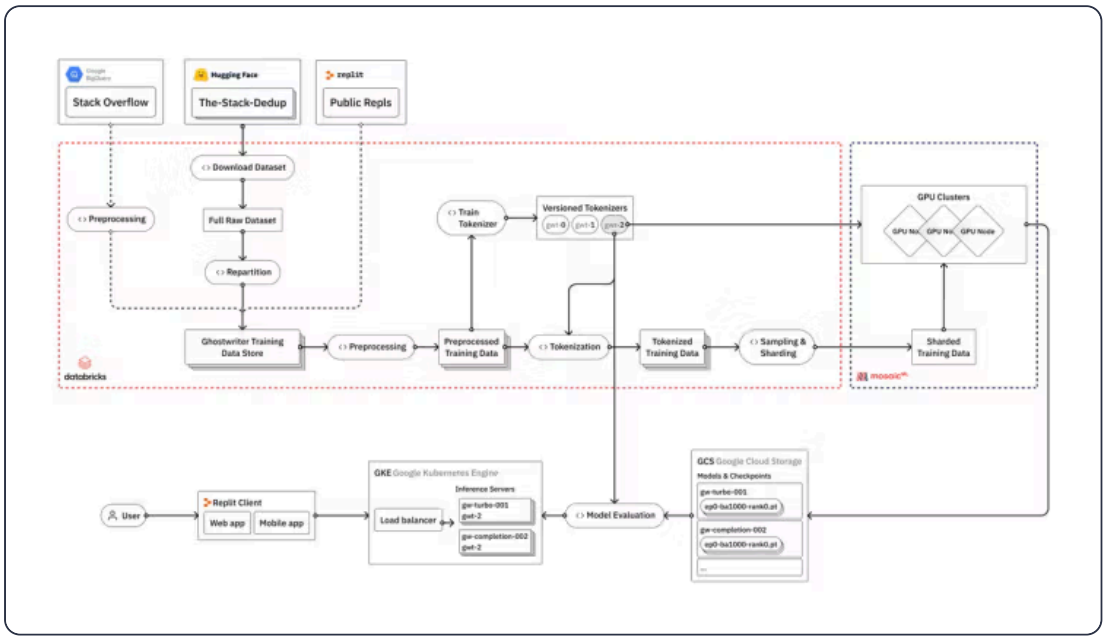
AI Engineering Infrastructure

How to train your own Large Language Models



Tue, Apr 18, 2023

Updated at: Thu, Jan 2, 2025

The AI Team @ Replit

Learn how Replit trains Large Language Models (LLMs) using Databricks, Hugging Face, and MosaicML

Introduction

Large Language Models, like OpenAI's GPT-4 or Google's PaLM, have taken the world of artificial intelligence by storm. Yet most companies don't currently have the ability

to train these models, and are completely reliant on only a handful of large tech firms as providers of the technology.

At Replit, we've invested heavily in the infrastructure required to train our own Large Language Models from scratch. In this blog post, we'll provide an overview of how we train LLMs, from raw data to deployment in a user-facing production environment. We'll discuss the engineering challenges we face along the way, and how we leverage the vendors that we believe make up the modern LLM stack: Databricks, Hugging Face, and MosaicML.

While our models are primarily intended for the use case of code generation, the techniques and lessons discussed are applicable to all types of LLMs, including general language models. We plan to dive deeper into the gritty details of our process in a series of blog posts over the coming weeks and months.

Why train your own LLMs?

One of the most common questions for the AI team at Replit is "why do you train your own models?" There are plenty of reasons why a company might decide to train its own LLMs, ranging from data privacy and security to increased control over updates and improvements.

At Replit, we care primarily about customization, reduced dependency, and cost efficiency.

- **Customization**. Training a custom model allows us to tailor it to our specific needs and requirements, including platform-specific capabilities, terminology, and context that will not be well-covered in general-purpose models like GPT-4 or even code-specific models like Codex. For example, our models are

trained to do a better job with specific web-based languages that are popular on Replit, including Javascript React (JSX) and Typescript React (TSX).

- **Reduced dependency**. While we'll always use the right model based on the task at hand, we believe there are benefits to being less dependent on only a handful of AI providers. This is true not just for Replit but for the broader developer community. It's why we plan to open source some of our models, which we could not do without the means to train them.
- **Cost efficiency**. Although costs will continue to go down, LLMs are still prohibitively expensive for use amongst the global developer community. At Replit, our mission is to bring the next billion software creators online. We believe that a student coding on their phone in India should have access to the same AI as a professional developer in Silicon Valley. To make this possible, we train custom models that are smaller, more efficient, and can be hosted with drastically reduced cost.

Data pipelines

LLMs require an immense amount of data to train. Training them requires building robust data pipelines that are highly optimized and yet flexible enough to easily include new sources of both public and proprietary data.

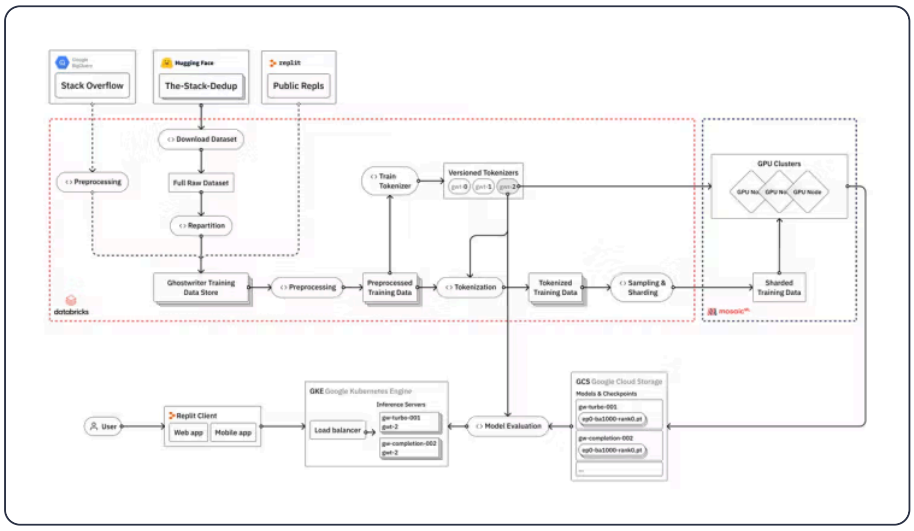
The Stack

We begin with The Stack as our primary data source which is available on Hugging Face. Hugging Face is a great resource for datasets and pre-trained models. They also provide a variety of useful tools as part of the

Transformers library, including tools for tokenization, model inference, and code evaluation.

The Stack is made available by the BigCode project. Details of the dataset construction are available in Kocetkov et al. (2022). Following de-duplication, version 1.2 of the dataset contains about 2.7 TB of permissively licensed source code written in over 350 programming languages.

The Transformers library does a great job of abstracting away many of the challenges associated with model training, including working with data at scale. However, we find it insufficient for our process, as we need additional control over the data and the ability to process it in distributed fashion.



llm-training

Data processing

When it comes time for more advanced data processing, we use Databricks to build out our pipelines. This approach also makes it easy for us to introduce additional data sources (such as Replit or Stack Overflow) into our process, which we plan to do in future iterations.

The first step is to download the raw data from Hugging Face. We use Apache Spark to parallelize the dataset builder process across each programming language. We then repartition the data and rewrite it out in parquet format with optimized settings for downstream processing.

Next, we turn to cleaning and preprocessing our data. Normally, it's important to deduplicate the data and fix various encoding issues, but The Stack has already done this for us using a near-deduplication technique outlined in Kocetkov et al. (2022). We will, however, have to rerun the deduplication process once we begin to introduce Replit data into our pipelines. This is where it pays off to have a tool like Databricks, where we can treat The Stack, Stackoverflow, and Replit data as three sources within a larger data lake, and utilize them as needed in our



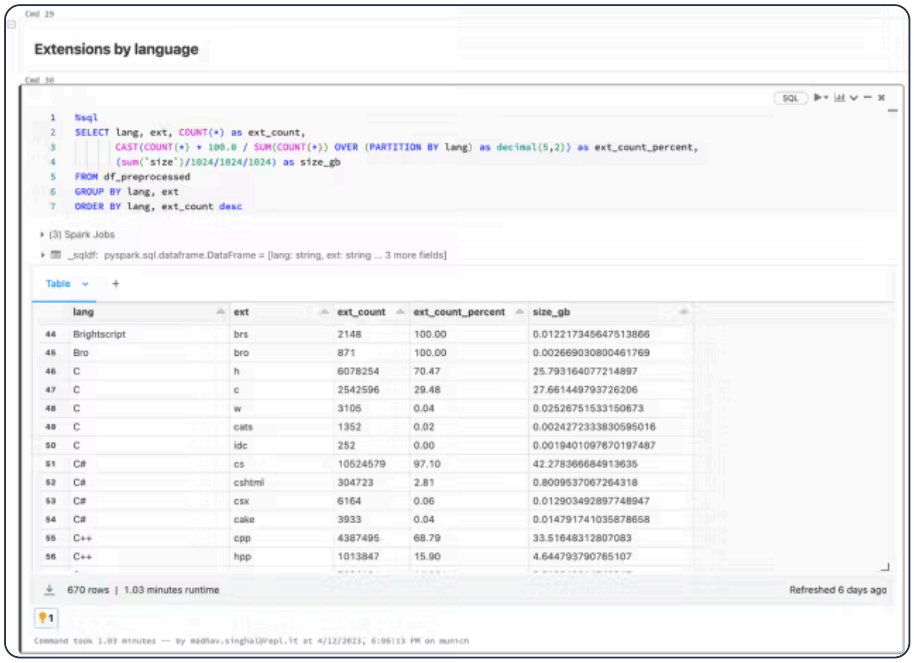
Agent 4

An additional benefit of using Databricks is that we can run scalable and tractable analytics on the underlying data. We run all types of summary statistics on our data sources, check long-tail distributions, and diagnose any issues or inconsistencies in the process. All of this is done within Databricks notebooks, which can also be integrated with MLFlow to track and reproduce all of our analyses along the way. This step, which amounts to taking a periodic x-ray of our data, also helps inform the various steps we take for preprocessing.

For preprocessing, we take the following steps:

- We anonymize the data by removing any Personal Identifiable Information (PII), including emails, IP addresses, and secret keys.
- We use a number of heuristics to detect and remove auto-generated code.

- For a subset of languages, we remove code that doesn't compile or is not parseable using standard syntax parsers.
- We filter out files based on average line length, maximum line length, and percentage of alphanumeric characters.



Extensions by language

```
%sql
SELECT lang, ext, COUNT(*) as ext_count,
       CAST(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (PARTITION BY lang) as decimal(5,2)) as ext_count_percent,
       (sum(`size`)/1024/1024/1024) as size_gb
FROM df_preprocessed
GROUP BY lang, ext
ORDER BY lang, ext_count desc
```

(3) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [lang: string, ext: string ... 3 more fields]

Table +

	lang	ext	ext_count	ext_count_percent	size_gb
44	Brightscript	brs	2148	100.00	0.012217345647513866
45	Bro	bro	871	100.00	0.002669030800461769
46	C	h	6078254	70.47	25.793164077214897
47	C	c	2542596	29.48	27.661449793726206
48	C	w	3105	0.04	0.025267515331506 73
49	C	cats	1352	0.02	0.0024272333830595016
50	C	idc	252	0.00	0.0019401097670197487
51	C#	cs	10524579	97.10	42.278366684913635
52	C#	cshtml	304723	2.81	0.800953706726431 8
53	C#	csx	6164	0.06	0.012903492897748947
54	C#	cake	3933	0.04	0.014791741035878658
55	C++	cpp	4387495	68.79	33.51648312807083
56	C++	hpp	1013847	15.90	4.644793790765107

670 rows | 1.03 minutes runtime

Refreshed 6 days ago

the-stack-db-notebook

Tokenization and vocabulary training

Prior to tokenization, we train our own custom vocabulary using a random subsample of the same data that we use for model training. A custom vocabulary allows our model to better understand and generate code content. This results in improved model performance, and speeds up model training and inference.

This step is one of the most important in the process, since it's used in all three stages of our process (data pipelines, model training, inference). It underscores the importance of having a robust and fully-integrated infrastructure for your model training process.

We plan to dive deeper into tokenization in a future blog post. At a high-level, some important things we have to account for are vocabulary size, special tokens, and reserved space for sentinel tokens.

Once we've trained our custom vocabulary, we tokenize our data. Finally, we construct our training dataset and write it out to a sharded format that is optimized for feeding into the model training process.

Model training

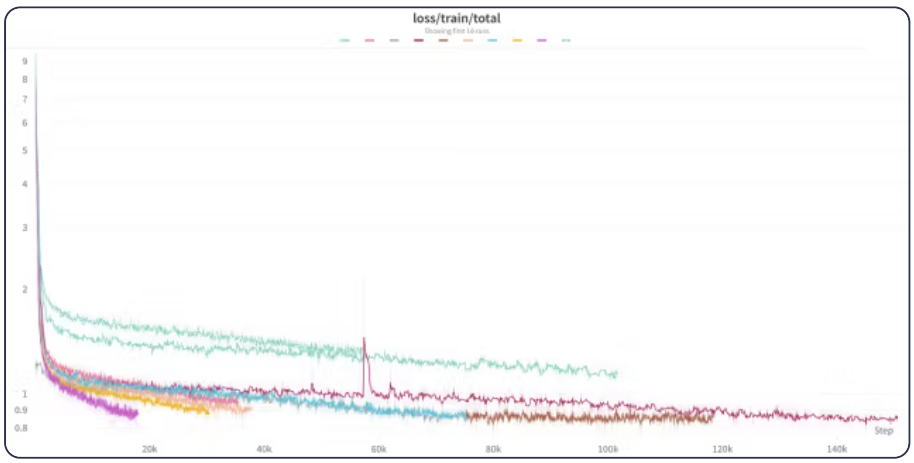
We train our models using MosaicML. Having previously deployed our own training clusters, we found that the MosaicML platform gives us a few key benefits.

- **Multiple cloud providers**. Mosaic gives us the ability to leverage GPUs from different cloud providers without the overhead of setting up an account and all of the required integrations.
- **LLM training configurations**. The Composer library has a number of well-tuned configurations for training a variety of models and for different types of training objectives.
- **Managed infrastructure**. Their managed infrastructure provides us with orchestration, efficiency optimizations, and fault tolerance (i.e., recovery from node failures).

In determining the parameters of our model, we consider a variety of trade-offs between model size, context window, inference time, memory footprint, and more. Larger models typically offer better performance and are more capable of transfer learning. Yet these models have higher computational requirements for both training and inference. The latter is especially important to us. Replit is

a cloud native IDE with performance that feels like a desktop native application, so our code completion models need to be lightning fast. For this reason, we typically err on the side of smaller models with a smaller memory footprint and low latency inference.

In addition to model parameters, we also choose from a variety of training objectives, each with their own unique advantages and drawbacks. The most common training objective is next token prediction. This typically works well for code completion, but fails to take into account the context further downstream in a document. This can be mitigated by using a "fill-in-the-middle" objective, where a sequence of tokens in a document are masked and the model must predict them using the surrounding context. Yet another approach is UL2 (Unsupervised Latent Language Learning), which frames different objective functions for training language models as denoising tasks, where the model has to recover missing sub-sequences of a given input.



loss-curves

Once we've decided on our model configuration and training objectives, we launch our training runs on multi-node clusters of GPUs. We're able to adjust the number of nodes allocated for each run based on the size of the model we're training and how quickly we'd like to complete the training process. Running a large cluster of

GPUs is expensive, so it's important that we're utilizing them in the most efficient way possible. We closely monitor GPU utilization and memory to ensure that we're getting maximum possible usage out of our computational resources.

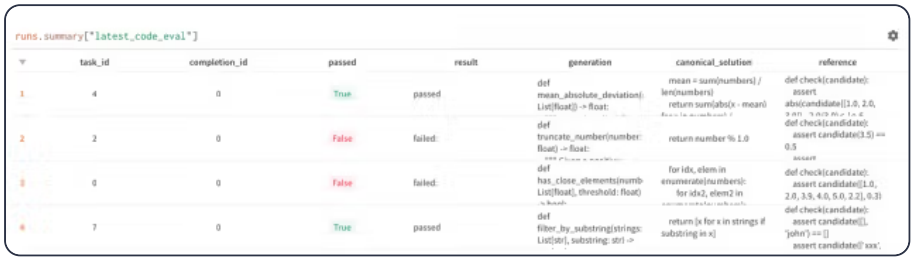
We use Weights & Biases to monitor the training process, including resource utilization as well as training progress. We monitor our loss curves to ensure that the model is learning effectively throughout each step of the training process. We also watch for loss spikes. These are sudden increases in the loss value and usually indicate issues with the underlying training data or model architecture. Because these occurrences often require further investigation and potential adjustments, we enforce data determinism within our process, so we can more easily reproduce, diagnose, and resolve the potential source of any such loss spike.

Evaluation

To test our models, we use a variation of the HumanEval framework as described in Chen et al. (2021). We use the model to generate a block of Python code given a function signature and docstring. We then run a test case on the function produced to determine if the generated code block works as expected. We run multiple samples and analyze the corresponding Pass@K numbers.

This approach works best for Python, with ready to use evaluators and test cases. But because Replit supports many programming languages, we need to evaluate model performance for a wide range of additional languages. We've found that this is difficult to do, and there are no widely adopted tools or frameworks that offer a fully comprehensive solution. Two specific challenges include conjuring up a reproducible runtime

environment in any programming language, and ambiguity for programming languages without widely used standards for test cases (e.g., HTML, CSS, etc.). Luckily, a "reproducible runtime environment in any programming language" is kind of our thing here at Replit! We're currently building an evaluation framework that will allow any researcher to plug in and test their multi-language benchmarks. We'll be discussing this in a future blog post.



humaneval-results

Deployment to production

Once we've trained and evaluated our model, it's time to deploy it into production. As we mentioned earlier, our code completion models should feel fast, with very low latency between requests. We accelerate our inference process using NVIDIA's FasterTransformer and Triton Server. FasterTransformer is a library implementing an accelerated engine for the inference of transformer-based neural networks, and Triton is a stable and fast inference server with easy configuration. This combination gives us a highly optimized layer between the transformer model and the underlying GPU hardware, and allows for ultra-fast distributed inference of large models.

Upon deploying our model into production, we're able to autoscale it to meet demand using our Kubernetes infrastructure. Though we've discussed autoscaling in

previous blog posts, it's worth mentioning that hosting an inference server comes with a unique set of challenges. These include large artifacts (i.e., model weights) and special hardware requirements (i.e., varying GPU sizes/counts). We've designed our deployment and cluster configurations so that we're able to ship rapidly and reliably. For example, our clusters are designed to work around GPU shortages in individual zones and to look for the cheapest available nodes.

Before we place a model in front of actual users, we like to test it ourselves and get a sense of the model's "vibes". The HumanEval test results we calculated earlier are useful, but there's nothing like working with a model to get a feel for it, including its latency, consistency of suggestions, and general helpfulness. Placing the model in front of Replit staff is as easy as flipping a switch. Once we're comfortable with it, we flip another switch and roll it out to the rest of our users.



monitoring

We continue to monitor both model performance and usage metrics. For model performance, we monitor metrics like request latency and GPU utilization. For usage, we track the acceptance rate of code suggestions and break it out across multiple dimensions including

programming language. This also allows us to A/B test different models, and get a quantitative measure for the comparison of one model to another.

Feedback and iteration

Our model training platform gives us the ability to go from raw data to a model deployed in production in less than a day. But more importantly, it allows us to train and deploy models, gather feedback, and then iterate rapidly based on that feedback.

It's also important for our process to remain robust to any changes in the underlying data sources, model training objectives, or server architecture. This allows us to take advantage of new advancements and capabilities in a rapidly moving field where every day seems to bring new and exciting announcements.

Next, we'll be expanding our platform to enable us to use Replit itself to improve our models. This includes techniques such as Reinforcement Learning Based on Human Feedback (RLHF), as well as instruction-tuning using data collected from Replit Bounties.

Next steps

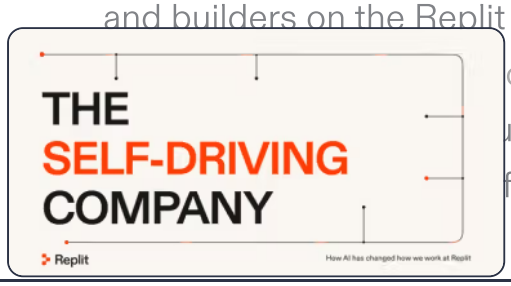
While we've made great progress, we're still in the very early days of training LLMs. We have tons of improvements to make and lots of difficult problems left to solve. This trend will only accelerate as language models continue to advance. There will be an ongoing set of new challenges related to data, algorithms, and model evaluation.

If you're excited by the many engineering challenges of training LLMs, we'd love to speak with you. We love feedback, and would love to hear from you about what we're missing and what you would do differently.

Follow @Replit

More

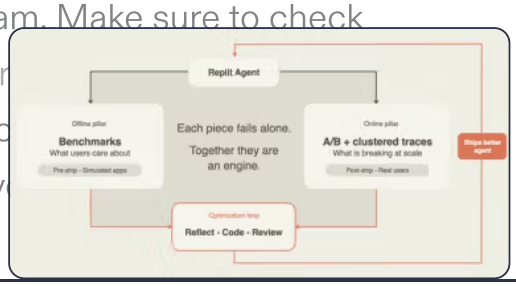
We're always looking for talented engineers, researchers, and builders on the Replit AI team. Make sure to check career
u can c
from y



Thu, Jul 16, 2026

The Self-Driving Company

We are beginning to see what happens when a company learns to operate itself.



Tue, Jun 23, 2026

Closing the loop: Evaluating and improving Replit Agent at scale

Most Replit Agent users start with an idea. They describe the goal in natural language —...



Thu, Feb 26, 2026

We Built a Video Rendering Engine by Lying to the Browser About What Time It Is

How Replit turns any web page into a deterministic video by virtualizing time itself, patchin...

Replit

CA
6 58 AM

Made in sunny California.

All rights reserved. Copyright © 2026 Replit, Inc.

HANDY LINKS

About us
Vibe Coding 101
Help
How to guides
Import from GitHub
Status
Additional resources
Brand kit
Certifications
Partnerships

LEGAL

Terms of service
Commercial agreement
Privacy
Subprocessors
DPA
Report abuse

CONNECT

X / Twitter
Tiktok
Facebook
Instagram
Linkedin